Appening AI intern Assignment

AI Engineer Interview Task: LangGraph & Pinecone RAG Chatbot

Role: AI Engineer
Focus: Custom Python Implementation (No Code/Low-Code Platforms Avoided)
Deadline: Within 24 hours from email receipt
Submission: Public GitHub Repository Link

---

1. Project Objective

Develop a robust Retrieval-Augmented Generation (RAG) chatbot using **LangGraph**, **Pinecone** (or alternative vector database), and vector embeddings. The chatbot must answer queries strictly based on the provided document:
Knowledge Base PDF: Agentic AI eBook

---

2. System Architecture Overview

The system follows a cyclic, graph-based RAG workflow built with LangGraph to perform content ingestion, chunking, vector retrieval, and strictly grounded response generation.

Component	Technology	Responsibility
Data Ingestion	PyPDF / LangChain TextSplitter	Download eBook PDF, chunk text into standard segments with overlap.
Embeddings & Vector DB	OpenAI Embeddings + Pinecone	Convert text chunks into dense vector representations and index for fast semantic lookup.
Orchestration Graph	LangGraph	Define nodes for retrieval, hallucination grading, and response generation state graphs.

Component	Technology	Responsibility
API & Interface	FastAPI or Streamlit	Expose endpoint returning response payload containing answer, context, and score.

---

3. Key Requirements & Technical Expectations

A. Ingestion & Vector Storage Pipeline

- Download and parse Ebook-Agentic-AI.pdf.
- Chunk text effectively (e.g., chunk size 500–1000 characters with 100 overlap).
- Generate embeddings and store vectors with metadata (source text chunk, page number) in Pinecone.

B. LangGraph RAG Workflow

- **Retrieve Node:** Query Pinecone index for top-k similar chunks.
- **Generate Node:** Synthesize grounded answer using an LLM based ONLY on retrieved context.
- **Groundedness / Confidence Scoring:** Compute confidence score or verify answer grounding to prevent hallucinations.

C. API/UI Output Payload

Every API response or UI output must return a structured JSON response formatted as:

```
{
  "query": "What is Agentic AI?",
  "final_answer": "Agentic AI refers to autonomous systems that...",
  "retrieved_context_chunks": [
    "Chunk 1 text from PDF...",
    "Chunk 2 text from PDF..."
  ],
  "confidence_score": 0.92
}
```

---

4. Sample Queries for Validation & Testing

1. **Definition & Scope:** What is the core definition of Agentic AI as outlined in the eBook?
2. **Architecture & Paradigms:** What are the main architectural components required to build agentic systems?
3. **Use Cases:** What real-world industry use cases for Agentic AI are discussed in the

eBook?

4. **Comparison:** How does Agentic AI differ from traditional generative AI chatbots according to the text?
5. **Challenges & Considerations:** What key challenges or limitations of Agentic AI are mentioned in the document?
6. **Out-of-Scope Test (Groundedness Check):** What is the capital of France? *(Expected behavior: Refuse to answer or state that information is not available in the eBook context.)*

---

5. Submission Deliverables Checklists

- [ ] Public GitHub Repository created.
- [ ] README.md with setup guide, requirement installation, and architecture breakdown.
- [ ] Functional ingestion module and LangGraph RAG pipeline.
- [ ] API (FastAPI/Flask) or Web UI (Streamlit/Gradio).
- [ ] Verified response structure containing final_answer, retrieved_context_chunks, and confidence_score.